Engineering, Built Environment and IT
Department of Computer Science

Programming Languages
COS 333

Semester Test 2
9 May 2025

Examiner: Mr W. S. van Heerden

Instructions:

1. Read the questions carefully and answer all questions.
2. This section comprises of **23** questions, and a total of **35** marks.
3. You have **1.5** hours to complete this test.
4. This test is an *online* assessment: You will be instructed to submit your test when the time (1.5 hours) expires. You can also submit the test earlier if you are done ahead of time.
5. This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.

- You are not allowed to consult any sources other than the Practical 3 and 4 specifications and the SWI-Prolog Reference Manual (provided as documentation for Practical 3).
- For practical implementation questions, your Prolog submission must be working Prolog code that will successfully interpret using version 9.0.4 of the SWI_Prolog interpreter. You may (and should) use this interpreter to test your submissions. Once you have finished writing each program, make sure each is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.
- You may use a non-programmable calculator.
- You may not use any other programmable devices.

Integrity Statement:
The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Question 1

1 Point

Consider the following Prolog proposition implementation:

```
tail([], []).
tail([H|T], T).
```

The `tail` proposition succeeds if the second parameter is the tail of the first parameter, where the proposition assumes that the tail of an empty list is an empty list. This implementation results in a Prolog interpreter raising a warning. Consider the following statements about the `tail` proposition, and **select** the one that is most correct.

- The warning can be avoided by replacing H with an _ character, although the tail proposition will not work correctly for certain queries.
- The warning can be avoided by replacing H with an _ character, although the tail proposition will be less efficient than the original implementation provided above.
- The warning can be avoided by replacing H with an _ character, although the tail proposition will be less reliable than the original implementation provided above.
- The warning can be avoided without changing how the tail proposition works by replacing H with an _ character.

Question 2

1 Point

Consider the following Prolog proposition implementation:

```
primeGreaterThanOneMillion(X) :- prime(X), X > 1000000.
```

Assume that the proposition `prime(X)` succeeds if `X` is a prime number. Consider the following statements about the `primeGreaterThanOneMillion` proposition, and **select** the one that is most correct.

- The `primeGreaterThanOneMillion` proposition will execute more efficiently if it is rewritten as:

  ```
  primeGreaterThanOneMillion(X) :- X > 1000000, prime(X).
  ```

- The `primeGreaterThanOneMillion` proposition will execute less efficiently if it is rewritten as:

  ```
  primeGreaterThanOneMillion(X) :- X > 1000000, prime(X).
  ```

- The `primeGreaterThanOneMillion` proposition will not work correctly unless it is rewritten as:

  ```
  primeGreaterThanOneMillion(X) :- X > 1000000, prime(X).
  ```

- The `primeGreaterThanOneMillion` proposition cannot be successfully implemented, due to a deficiency of Prolog.

Question 3

1 Point

Assume that you wish to write a Prolog proposition that ensures that two variables, `X` and `Y`, do not represent the same thing. Consider the following statements, and **select** the one that describes the most rigorous and correct way to achieve this.

- X and Y are always different because `X` and `Y` represent two distinct atoms.
- X and Y are always different because `X` and `Y` represent two distinct variables.
- By defining a proposition called `unequal(X, Y)` for every pair of atoms that are not the same.
- By using the proposition `not(X = Y)`.

Question 4

5 Points

Assume you have Prolog facts describing family relationships, taking the following form:

```
father(bill, jake).
father(bill, shelley).
father(jake, ted).
father(ron, liz).

mother(mary, jake).
mother(mary, shelley).
mother(janet, ted).
mother(shelley, liz).
```

Note that the **only** facts you are allowed to define are the `father` and `mother` propositions. If you define any facts using additional propositions, you will **forfeit all marks for this question**. The proposition `father(X, Y)` means that person `X` is the father of person `Y`, and `mother(X, Y)` means that person `X` is the mother of person `Y`. Note that it is possible for single parents to exist (i.e. a person may have only a mother or only a father). Same sex parents are also possible (i.e. a person may have two mothers or two fathers).

Write a Prolog proposition called `nephewNiece(X, Y)`, which succeeds if person `X` is the nephew or niece of person `Y`. A person's nephew or niece is the son or daughter, respectively, of that person's brother or sister. In the above example, `shelley` is the sister of `jake` because `shelley` and `jake` share at least one parent (in this example, `bill` or `mary`). This means that `ted` is the nephew of `shelley`, because `ted` is the son of `jake`. Similarly, `liz` is the niece of `jake`, because `liz` is the son of `shelley`.

Hint: When testing your proposition, use variables, such as in the query `nephewNiece(X, Y)`, then use Prolog's support for re-querying to determine all the pairs of atoms that satisfy the `nephewNiece` proposition, and ensure that the proposition does not succeed for atoms that do not satisfy the properties of nephews and nieces.

Submit your program code in a file named `u99999999.pl` (where `99999999` is your student number) to the assignment submission slot labelled "Semester Test 2: Simple Prolog Proposition", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below.

Take careful note of the following requirements:

- Your submission must be working Prolog code that will be successfully interpreted by the SWI-Prolog interpreter installed on the Windows computers in the Informatorium. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3. Once you have finished writing your program, save it in a text file with the appropriate name, and submit it to the correct upload slot.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, the `is` operator, cuts, and the built-in `not` proposition discussed in the textbook and slides**. In particular, do not use if-then, if-then-else, and similar constructs. Also, do not use the logical and operator (represented using a semicolon symbol). You may NOT use any more complex predicates provided by the Prolog system itself (including the built-in `member`, `append`, and `reverse` propositions). In other words, you must write all your own propositions, other than `not`. **Failure to observe this rule will result in all marks for this question being forfeited**.

True

False

Question 5

5 Points

Write a Prolog proposition called `getAbsNonZeros(L1, L2)`, where `L1` and `L2` are both simple numeric lists (i.e. lists containing only integers). The `getAbsNonZeros` proposition succeeds if `L2` contains the absolute values of only the non-zero elements contained in `L1`, in their original order (zero values contained in `L1` are not included in `L2`). For example, the query:

```
?- getAbsNonZeros([0], X).
```

should be true with the instantiation `X = []`. This is because there are no non-zero values in the list `[0]`. Similarly, the query

```
?- getAbsNonZeros([1, 0, -2, 0], X).
```

should be true with the instantiation `X = [1, 2]`. This is because the absolute values of 1 and -2 are 1 and 2, respectively, while the second and fourth elements are zero values, and are therefore not included in `X`.

Submit your program code in a file named `u99999999.pl` (where `99999999` is your student number) to the assignment submission slot labelled "Semester Test 2: List Processing Prolog Proposition", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below.

Take careful note of the following requirements:

- Your submission must be working Prolog code that will be successfully interpreted by the SWI-Prolog interpreter installed on the Windows computers in the Informatorium. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3. Once you have finished writing your program, save it in a text file with the appropriate name, and submit it to the correct upload slot.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, the `is` operator, cuts, and the built-in `not` proposition discussed in the textbook and slides**. In particular, do not use if-then, if-then-else, and similar constructs. Also, do not use the logical and operator (represented using a semicolon symbol). You may NOT use any more complex predicates provided by the Prolog system itself (including the built-in `member`, `append`, and `reverse` propositions). In other words, you must write all your own propositions, other than `not`. **Failure to observe this rule will result in all marks for this question being forfeited**.

True

False

Question 6

1 Point

Consider the following programming languages, and **select** the one that is the least reliable in terms of name length.

- Fortran
- C99
- C++
- C#

Question 7

1 Point

Consider the following statements about variable names in Perl and Java, and **select** the one that is the most correct.

- Variable names in Perl have a higher cost of maintenance than variable names in Java.
- Variable names in Perl are more writable than variable names in Java.
- Variable names in Perl are more readable than variable names in Java.
- Variable names in Perl are less reliable than variable names in Java.

Question 8

1 Point

Consider the following options, and **select** the time at which a name is bound to a stack-dynamic variable.

- Language design time.
- Language implementation time.
- Compile time.
- Load time.
- Runtime.

Question 9

1 Point

In Fortran I, a variable with a names that starts with `I`, `J`, `K`, `L`, `M`, and `N` is automatically an integer variable. Consider the following options, and **select** the one that most correctly describes the kind of type binding that takes place for this variable.

- Explicit type declaration.
- Implicit type declaration without type inferencing.
- Implicit type declaration with type inferencing.
- Dynamic type binding.

Question 10

1 Point

Question 10

Consider the following program code in a hypothetical programming language:

```
f() {
   myValue = 10
   print(myValue)
   myValue = myValue + 5
}

main() {
   f()
   f()
   f()
}
```

Assume that program execution starts with the `main` subprogram, and that the output of the program is:

```
10 15 20
```

Consider the following options, and **select** the one that correctly describes the category of the variable `myValue`, according to lifetime.

- Static variable.
- Stack-dynamic variable.
- Explicit heap-dynamic variable.
- Implicit heap-dynamic variable.

Question 11

1 Point

Assume a hypothetical programming language uses dynamic type binding. Consider the following options, and **select** the one that most correctly describes the category of the variables in this programming language, according to lifetime.

- Static variables.
- Stack-dynamic variables.
- Explicit heap-dynamic variables.
- Implicit heap-dynamic variables.

Question 12

2 Points

Consider the following program code in a hypothetical programming language:

```
void main() {

  void doSomething() {
      print(X)
  }

  var X = 10

  call doSomething()
}
```

Assume that the programming language implements variable hiding, that variable scope ranges from the variable declaration to the end of the block in which the variable appears, that nested subprograms are supported, and that execution starts with the `main` subprogram. **Provide** only the output of the above program code assuming the following scoping rules are used by the programming language. Provide only the numeric output (do not include additional output such as spaces or quotes) if output is successfully generated, and the text "Error" (without the quotes) if an error or invalid output is produced.

Static scoping:

Dynamic scoping:

Question 13

2 Points

Consider the following program code in a hypothetical programming language (line numbers are provided only for reference purposes):

```
1. void f()
2.     allocateInteger(myValue, 15)
3.     print(myValue)

4. int main()
5.     f()
6.     return 0
```

Assume that the programming language uses static scope, that variables are not explicitly declared, that the `main` subprogram is the first thing to be executed when the program runs, and that indentation indicates the body of subprograms. **Fill in** the start and end of the lifetime of the variable `myValue`, using only line numbers (do not include full stops or other punctuation).

Lifetime of `myValue`: From

up to and including

Question 14

1 Point

Identify the kind of named constants that are used in C++.

Question 15

1 Point

Some programming languages support a primitive data type that represents a complex number. Consider the following statements about support for primitive data types representing complex numbers, and **select** the one that is most correct.

- Support for a primitive data type that represents complex numbers improves the writability of a programming language.
- Support for a primitive data type that represents complex numbers improves the readability of a programming language.
- Support for a primitive data type that represents complex numbers improves the execution cost of a programming language.
- Support for a primitive data type that represents complex numbers improves the reliability of a programming language.

Question 16

1 Point

Consider the following statements about the `String` and `StringBuffer` classes in Java, and **select** the one that is most correct.

- A `String` object is less readable than a `StringBuffer` object.
- A `String` object is less writable than a `StringBuffer` object.
- A `String` object is less reliable than a `StringBuffer` object.
- A `String` object has a lower cost of execution than a `StringBuffer` object.

Question 17

2 Points

Consider the following program code in a hypothetical programming language:

```
main() {
   constant integer LENGTH = 3
   integer myArray[LENGTH]

   myArray[0] = 6
   myArray[1] = 2
   myArray[2] = 5

   print(myArray[2])
}
```

Assume that the length of `myArray` cannot be changed after it has been set. Consider the following options, and **select** only the ones that could possibly correctly describe the category of `myArray`. Note that incorrectly selected options will be penalised.

- ○ Static
- ○ Fixed stack-dynamic
- ○ Stack-dynamic
- ○ Fixed heap-dynamic
- ○ Heap-dynamic

Select up to 5 options

Question 18

1 Point

Assume that you wish to simulate the functionality of a associative array in a programming language that does not directly support associative arrays. Consider the following options, and **select** the one that will correctly simulate the functionality of an associative array in the programming language.

- ○ Using a normal array in a statically typed programming language, which stores values of different primitive data types.
- ○ Using a normal array in a statically typed programming language, which stores instances of an `Object` class, where all other classes inherit from the `Object` class.
- ○ Using a normal array in a statically typed programming language, where the array stores a key and a value at each subscript.
- ○ Using two normal arrays in a statically typed programming language, where one stores keys and the other stores corresponding values.

Question 19

1 Point

Consider the following statements about records in COBOL and Ada, and **select** the one that is most correct.

- References to record fields in COBOL are less writable than references to record fields in Ada.
- Records in COBOL have a lower cost of maintenance than records in Ada.
- Records in COBOL are less orthogonal than records in Ada.
- Records in COBOL are more orthogonal than records in Ada.

Question 20

1 Point

Consider the following statements about unions, and **select** the one that is most correct.

- Free unions have a lower cost of maintenance than discriminated unions.
- Free unions are more orthogonal than discriminated unions.
- Free unions are less writable than discriminated unions.
- Free unions are more effective at saving memory than discriminated unions.

Question 21

1 Point

Consider the following statements about pointers and references in C++, Java, and C#, and **select** the one that is most correct.

- Pointers and references have the most reliable support in Java.
- Pointers and references have the most reliable support in C#.
- Pointers and references have the least writable support in C#.
- Pointers and references have the least writable support in C++.

Question 22

2 Points

C++ is not considered to be a strongly typed programming language. Consider the following language features, and **select** only the ones that contribute to C++ not being strongly typed. Note that incorrectly selected options will be penalised.

- Explicit type declarations.
- Void pointers.
- Casting a base class pointer to a derived class pointer.
- Casting a derived class pointer to a base class pointer.

Select up to 4 options

Question 23

1 Point

Consider the following statements about name type equivalence and structure type equivalence, and **select** the one that is most correct.

- Name type equivalence has a lower cost of execution than structure type equivalence.
- Name type equivalence is more writable than structure type equivalence.
- Name type equivalence is less readable than structure type equivalence.
- Name type equivalence is less reliable than structure type equivalence.